Angular Signals, Authentication Tokens & RxJS Operators

1. Angular Signals

What are Signals?

Signals are Angular's modern reactive state management system introduced in Angular 16. They help Angular automatically track changes in state and update the UI efficiently without manually triggering change detection.

Signals solve many problems related to: Unnecessary re-rendering Complex RxJS state management Manual subscriptions Performance issues **Main Features:** Reactive by default Fine-grained updates Easy syntax Better performance No manual subscription handling

```
import { signal } from '@angular/core';

const counter = signal(0);

// Reading value
console.log(counter());

// Setting value
counter.set(10);

// Updating based on previous value
counter.update(v => v + 1);

console.log(counter());
```

Signals Inside Angular Component

Signals can directly bind with Angular templates. Whenever signal changes, UI updates automatically.

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-counter',
  template: `
      <h1>{{ count() }}</h1>

      <button (click)="increment()">
          Increment
      </button>
  `
})
export class CounterComponent {

   count = signal(0);

   increment() {
      this.count.update(v => v + 1);
   }
}
```

Computed Signals

Computed signals create derived values from existing signals. They automatically recalculate whenever dependent signals change.

Advantages: No manual recalculation Efficient caching Reactive derived state

```
import { signal, computed } from '@angular/core';

const price = signal(500);
const quantity = signal(3);

const totalPrice = computed(() => {
   return price() * quantity();
});

console.log(totalPrice());

quantity.set(5);

console.log(totalPrice());
```

Effect in Signals

Effects execute automatically whenever related signal values change.

Used for: Logging API calls Local storage updates DOM manipulation

```
import { signal, effect } from '@angular/core';

const username = signal('Sanket');

effect(() => {
   console.log('Current user:', username());
});

username.set('Rahul');
```

Signal vs Observable

Signals Simpler syntax Automatic tracking No subscription needed Great for component state **Observables** Powerful async handling Streaming data HTTP/WebSocket handling Requires subscription

```
// Signal
const count = signal(0);

// Observable
const count$ = new Observable();
```

---

2. Access Token & Refresh Token

Authentication systems commonly use JWT (JSON Web Token).

Access Token Short-lived token Used for API authentication Sent with every request Usually expires quickly **Refresh Token** Long-lived token Used to generate new access tokens Improves security and user experience

```
User Login Flow:

1. User enters credentials
2. Backend validates user
3. Backend sends:
      - Access Token
      - Refresh Token
4. Angular stores tokens
```

```
5. Access token used in API requests
6. Access token expires
7. Refresh token generates new token
```

Angular JWT Authentication Example

Angular usually sends access token using Authorization header.

```
const token = localStorage.getItem('token');

this.http.get('api/users', {
   headers: {
      Authorization: `Bearer ${token}`
   }
});
```

Refresh Token Flow

When access token expires: User should not login again Frontend sends refresh token Backend validates refresh token New access token generated

```
this.http.post('/refresh-token', {
   refreshToken: storedRefreshToken
})
.subscribe((res:any) => {

   localStorage.setItem(
      'token',
      res.accessToken
   );

});
```

---

3. RxJS Subject

A Subject is both: Observable Observer It allows multicasting data to multiple subscribers simultaneously.

```
import { Subject } from 'rxjs';

const subject = new Subject<number>();

subject.subscribe(v => {
   console.log('Subscriber A:', v);
});

subject.subscribe(v => {
   console.log('Subscriber B:', v);
});

subject.next(10);
subject.next(20);
```

BehaviorSubject

BehaviorSubject stores the latest emitted value.

Important: Requires initial value New subscribers instantly receive latest value Commonly used for authentication state

```
import { BehaviorSubject } from 'rxjs';
```

```
const user = new BehaviorSubject('Guest');

user.subscribe(v => console.log(v));

user.next('Sanket');
```

ReplaySubject

ReplaySubject stores previous emitted values and replays them to new subscribers.

```
import { ReplaySubject } from 'rxjs';

const replay = new ReplaySubject(2);

replay.next(1);
replay.next(2);
replay.next(3);

replay.subscribe(v => console.log(v));

// Output:
// 2
// 3
```

AsyncSubject

AsyncSubject emits only the final value after completion.

```
import { AsyncSubject } from 'rxjs';

const subject = new AsyncSubject();

subject.subscribe(v => console.log(v));

subject.next(10);
subject.next(20);
subject.next(30);

subject.complete();

// Output:
// 30
```

---

mergeMap Operator

mergeMap maps source values into inner observables and executes them in parallel.

Use Cases: Multiple API calls Parallel processing Handling async operations

```
import { from } from 'rxjs';
import { mergeMap } from 'rxjs/operators';

from([1,2,3])
.pipe(
   mergeMap(id => {
      return fetch(`api/user/${id}`);
   })
)
.subscribe(res => {
   console.log(res);
});
```

---

forkJoin Operator

forkJoin waits for all observables to complete and then returns combined result.

Similar to Promise.all() in JavaScript.

```
forkJoin({
   users: this.http.get('/users'),
   products: this.http.get('/products'),
   orders: this.http.get('/orders')
})
.subscribe(res => {

   console.log(res.users);
   console.log(res.products);
   console.log(res.orders);

});
```

---

mergeMap vs switchMap

mergeMap Runs all requests Parallel execution No cancellation **switchMap** Cancels previous request Only latest request survives Useful for search APIs

```
searchInput.valueChanges.pipe(

   switchMap(value => {
      return this.http.get(`/search/${value}`);
   })

).subscribe();
```
